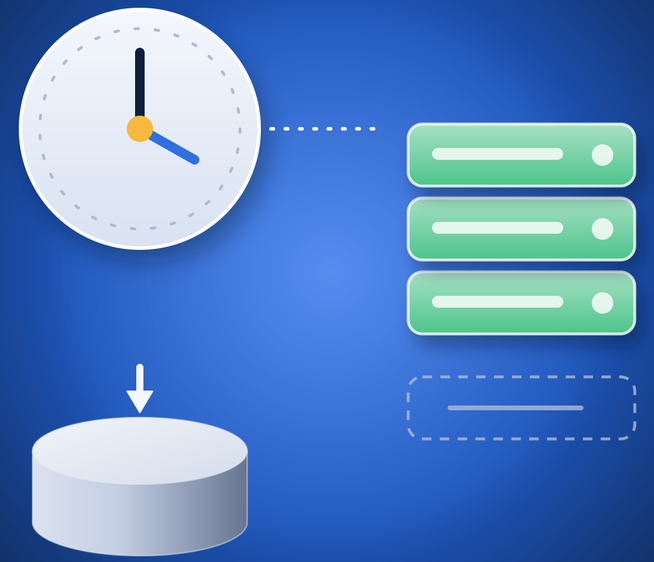
Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 10

Backups, Archives and cron

Lab 10 guide: package a project, prove it can be restored, keep only your own number of backups, and learn why cron is not your shell.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Create, list and restore a project archive, and tell archiving from compression.
2. Keep only the newest completed archives that this lab made, as many as your own number says, and nothing else.
3. Run the script in a bare environment, and check and clean up a practice cron job when cron is available.

THIS LAB

Lab 10

First the concepts, with examples. Then the lab on the server. Scheduling the real backup, log rotation and a health snapshot are optional extensions, so they come last.

DISCUSSION

You made a backup last week. Can you restore it?

A file that exists is not a backup that works. How would you find out before you need it?

CONCEPTS FIRST

Archives, retention and cron

Four ideas: tar joins files, restoring proves a backup, retention must stay narrow, and cron runs in a bare environment.

tar joins files, gzip shrinks them

student@server: ~/oslab-work/lab10

```
$ tar -cf demo.tar -C "$PWD" project
$ tar -tf demo.tar
$ tar -czf demo.tar.gz -C "$PWD" project
$ tar -tzf demo.tar.gz
```

tar combines files and folders into one archive. gzip compresses the bytes of that archive.

-c creates, -t lists, -x extracts, -f names the archive file, and -z adds gzip.

-C "$PWD" project stores the folder as project/..., so the paths inside do not depend on where you stood.

A very small archive can be larger when compressed, so no compression ratio is promised.

A backup is proven by restoring it

student@server: ~/oslab-work/lab10

```
$ mkdir -p restore-demo
$ tar -xzf demo.tar.gz -C restore-demo
$ cmp -- project/report.txt restore-demo/project/report.txt
$ cmp -- project/config.ini restore-demo/project/config.ini
```

Extract into a new, empty folder you own. Then compare each file with the original.

cmp prints nothing and returns 0 when the bytes are equal. Creating an archive without error does not show it can be restored.

CAREFUL

Only your own archives

Extract only archives you made. Never extract an unknown archive on the server.

Retention: keep a fixed number, touch nothing else

RULE

The policy

Keep the newest completed archives that the script made, as many as your keep value says. Remove older ones.

SCOPE

Narrow on purpose

Look only in your own backups/ folder. Consider only names like NAME-*.tar.gz, with your own name. Leave every other file alone.

HOW

A safe deletion

Sort the names, skip the newest ones you keep, and remove the rest. Quote every path. Never write rm $(ls ...).

What cron gives a script

Field	Range	Example
Minute	0 to 59	*/5 is every five minutes
Hour	0 to 23	2 is 02:00
Day of month	1 to 31	*
Month	1 to 12	*
Day of week	0 to 7	1 is Monday

A cron line has five time fields and a command. There is no year, so a date entry comes back every year unless something guards or removes it.

Cron starts the job in your home folder, with a small environment. Your shell's PATH and settings are not there.

Use absolute paths, and let the script find its own folder.

LAB 10

Backups you can restore

QuantumTech needs repeatable backups that work. Package a project, keep your own number of archives, and test under a bare environment.

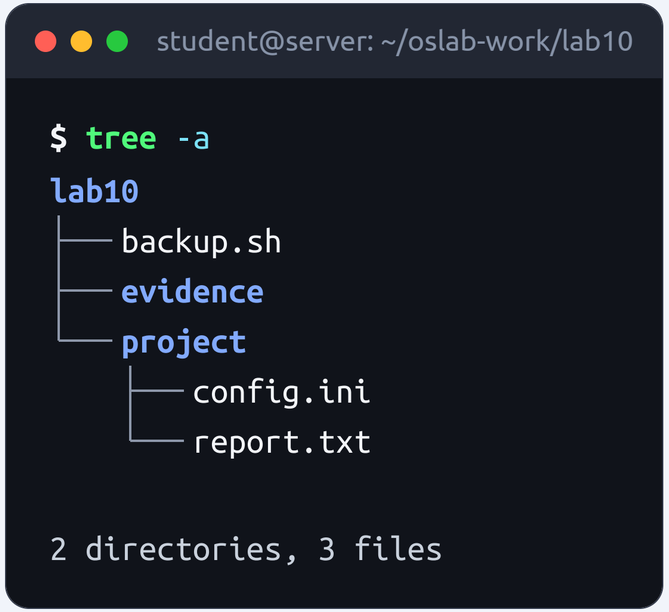
The scenario

SCENARIO

QuantumTech's backups

Package a small project, keep only a fixed number of completed archives that this lab made, and diagnose how the script runs in a restricted scheduling environment.

Your archive name and the number to keep are your own. This guide shows example values.



Inside the 120 minutes

Minutes	What happens
0 to 10	Setup
10 to 25	Guided: archiving and restoring
25 to 35	Prediction
35 to 55	Core 1: write the retention
55 to 70	Core 2: restricted environment and evidence

Minutes	What happens
70 to 85	Instructor demonstration, Plus and Challenge
85 to 100	Live checkpoint
100 to 110	Debrief
110 to 120	Submit

Six helper commands, and why you use them

Command	What it does
`oslab values lab10`	Shows your archive name and the number of archives to keep
`oslab predict lab10`	Saves your prediction once, before the experiment
`oslab hint lab10 1`	A hint, levels 1 to 3. Hints cost no points
`oslab check lab10`	Runs your backup.sh in a disposable folder and shows your milestones
`oslab checkpoint lab10`	Opens when the instructor says so, near the end
`oslab debrief lab10`	After the lab: you give the right answers to your prediction questions

Before the lab, run oslab doctor and oslab prelab lab10. It takes about ten minutes.

How you work

Core, Plus, Challenge

Core is for everyone. Plus and Challenge are for students who finish early. Full marks need only Core and the checkpoint.

AI tools

Allowed in Core, Plus and Challenge, so test every suggestion. Not allowed in the prediction and the checkpoint.

You write the retention

You receive a script that already makes a complete archive. You write the retention, and you prove that it deletes nothing else.

Delete only inside your own backups/ folder, only names that your script made. Never use a broad path, never scan your home folder, never edit /etc/crontab. A wrong prediction costs nothing.

Setup: start the lab

student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab start lab10
$ cd "$OSLAB_WORKSPACE/lab10"
$ mkdir -p evidence
$ oslab values lab10
```

- The first line sets OSLAB_WORKSPACE to ~/oslab-work, unless it is already set. Type it as it is.
- oslab start lab10 makes the lab folder.
- cd takes you into it, and mkdir -p makes the evidence folder.
- oslab values lab10 prints your archive name and your number.

Your values: a name and a number

The last command prints them

example: your values will be different

```
$ oslab values lab10
Your personal values for lab10 (your-account):
  keep     = 3
  name     = daily
```

What they mean

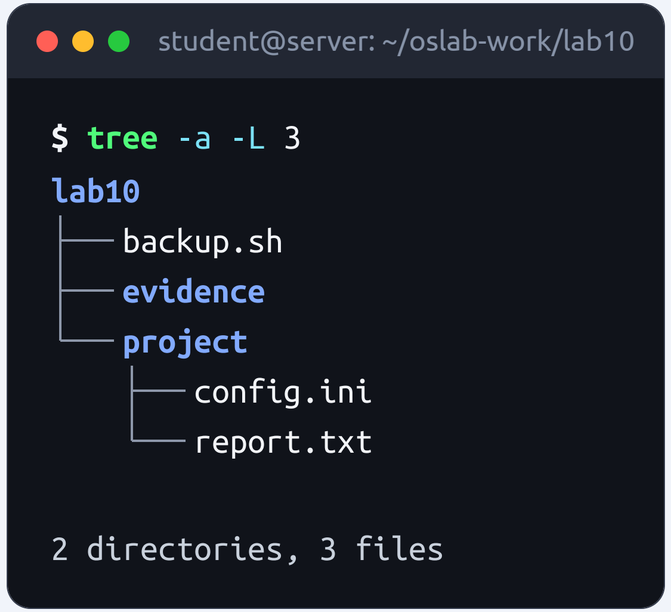
- name is the start of the name of every archive your script makes.
- keep is how many completed archives stay.
- Both are already written at the top of backup.sh. Write them down.

The capital words are placeholders

The instruction says	With the example values, you type
`backups/NAME-*.tar.gz`	`backups/daily-*.tar.gz`
keep the newest `KEEP` archives	keep the newest 3 archives

The instruction cannot know your values, so it writes NAME and KEEP where they belong. Never type the capital words. Type your own word and number.

Your folder



backup.sh makes a complete archive. The retention is your TODO.

project/ is the small project to back up: report.txt and config.ini.

oslab check lab10 now passes the archive part. The retention does not pass yet. The hidden .oslab-managed.json is the helper's. Leave it.

GUIDED

Archive and restore

Ten to twenty-five minutes, with the instructor. Make an archive, look inside, restore it elsewhere, and compare.

Step 1: look at the project

student@server: ~/oslab-work/lab10

```
$ find project -maxdepth 2 -type f
$ cat project/report.txt project/config.ini
$ mkdir -p backups
```

find lists the files of the project, down to two levels. cat shows what they hold.

mkdir -p backups makes the folder that will hold your archives.

Step 2: an archive, then a compressed one

student@server: ~/oslab-work/lab10

```
$ tar -cf demo.tar -C "$PWD" project
$ tar -tf demo.tar
$ tar -czf demo.tar.gz -C "$PWD" project
$ tar -tzf demo.tar.gz
$ ls -l demo.tar demo.tar.gz
```

-C changes the folder for tar first, so the names inside start with project/ wherever you are.

tar packs many files into one. gzip, the z, compresses its bytes. A very small archive can be larger than expected: do not count on a ratio.

Step 3: restore the demo and compare

student@server: ~/oslab-work/lab10

```
$ mkdir -p restore-demo
$ tar -xzf demo.tar.gz -C restore-demo
$ cmp -- project/report.txt restore-demo/project/report.txt
$ cmp -- project/config.ini restore-demo/project/config.ini
```

cmp prints nothing and ends with status 0 when the files are the same. Never unpack archives from other people on the server.

THINK

Observe

Is making an archive without an error the same as being able to restore it? What does -C make independent of your current folder?

PREDICT

Guess first, then try

Before you write the retention. Ten minutes.

The prediction

```
student@server: ~/oslab-work/lab10

$ oslab predict lab10
```

It asks four questions and one sentence, with your own name and number. Answer alone, without AI.

PREDICTION

The rules

The first answer is the one that counts. A wrong guess costs nothing. You compare it with the real result in the debrief.

CORE 1

Write the retention

Twenty minutes, from minute 35 to 55. After every run, only the newest KEEP complete archives named NAME stay in backups/.

Step 1: read your script

backup.sh (example values)

```
#!/usr/bin/env bash
set -euo pipefail
keep=3
name=daily
work=$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)
mkdir -p "$work/backups"
stamp=$(date -u +%Y%m%dT%H%M%S%N)
final="$work/backups/$name-$stamp-$$.tar.gz"
pending=$(mktemp "$work/backups/.pending-XXXXXX")
trap 'rm -f -- "$pending"' EXIT
tar -czf "$pending" -C "$work" project
tar -tzf "$pending" >/dev/null
mv -- "$pending" "$final"
# TODO: retain only the newest $keep completed $name-*.tar.gz files.
printf 'created %s\n' "$final"
```

Read it with cat -n backup.sh. The keep= and name= lines hold your own values.

work is the folder that holds the script, so it works from any folder. The archive is built under a temporary name and moved into place only when complete.

trap removes the temporary file if something fails. The orange TODO is your work: the retention.

Step 2: run it once, restore, and save the proof

example name: use your own

```
$ bash backup.sh
$ archive=$(find backups -maxdepth 1 -type f -name 'daily-*.tar.gz' | sort | tail -n 1)
$ mkdir -p restore-backup
$ tar -tzf "$archive" | tee evidence/restore.txt
$ tar -xzf "$archive" -C restore-backup
$ cmp -- project/report.txt restore-backup/project/report.txt
$ printf 'report comparison=%s\n' "$?" | tee -a evidence/restore.txt
$ cmp -- project/config.ini restore-backup/project/config.ini
$ printf 'config comparison=%s\n' "$?" | tee -a evidence/restore.txt
```

The find line finds the newest completed archive: sort puts the names in order, and tail -n 1 takes the last.

List it first, then extract and compare both files. tee saves a copy as evidence.

Step 3: your decision, the retention

Open nano backup.sh and write it at the TODO

- Look only at your own backups/ folder.
- Consider only the complete archives this script makes: NAME-STAMP-PID.tar.gz.
- Keep the newest $keep, remove the others, and leave every other name alone.
- Sort the names. No unquoted rm $(ls ...). Quote every path, and write rm -- "$path".

YOUR TURN

You write it

Nobody gives you the lines. Define the owned names before you delete. Loop only over the entries after the first $keep, with every path quoted.

STUCK?

Hints

oslab hint lab10 1, then 2, then 3. AI is allowed here: test every suggestion against the two files that are not yours.

Steps 4 and 5: test it with files that are not yours

example name: use your own

```
$ printf 'unrelated; preserve me\n' > backups/keep-me.txt
$ printf 'not an archive made by the script\n' > backups/daily-notes.tar.gz
$ for run in 1 2 3 4 5; do bash backup.sh; done
$ ls backups
$ find backups -maxdepth 1 -type f -name 'daily-[0-9]*.tar.gz' | wc -l
$ cat backups/keep-me.txt
```

One of the extra files looks like an archive name but is not one.

TRY

Check

You must count exactly your keep number of archives, and both files that are not yours must still be there.

The environment trap

Fifteen minutes, from minute 55 to 70. A scheduled job has another folder and almost no environment.

Step 1: run it under a bare environment

student@server: ~/oslab-work/lab10

```
$ cd "$HOME"
$ env -i HOME="$HOME" PATH=/usr/bin:/bin /bin/bash "$OSLAB_WORKSPACE/lab10/backup.sh"
$ cd "$OSLAB_WORKSPACE/lab10"
```

env -i starts the command with an empty environment, and then only HOME and PATH are set. This is close to what cron gives a job.

This tests the assumptions about the folder and the environment. It is not real cron. The script finds its project because it computes its own folder.

Step 2: save the evidence

example name: use your own

```
$ {
$   find backups -maxdepth 1 -type f
$   printf 'completed archive count='
$   find backups -maxdepth 1 -type f -name 'daily-[0-9]*.tar.gz' | wc -l
$   cat backups/keep-me.txt
$   printf 'restricted environment: done\n'
$ } > evidence/automation.txt
```

If you can use cron (see Plus), add what you saw. If you cannot, add a line that says cron was not available, so that nobody believes it was tested.

Steps 3 and 4: check, then explain

```
student@server: ~/oslab-work/lab10

$ oslab check lab10
```

The four Core milestones must show PASS: a restorable archive, the retention, the files that are not yours, and the restricted environment.

oslab check runs your backup.sh in a disposable folder: once, several times with extra files around, and under env -i from another folder. It does not use your own backups/.

REPORT

Write down for your report

How did you establish that the archive is recoverable? What stopped the retention from deleting your extra files? What did you not test?

What each case shows

Case	Expected evidence
Archive and restore	The member list, and restored files with the same bytes
Several backups in a row	Your keep number of completed archives, and the files that are not yours untouched
Another folder, bare environment	The backup still finds its source, and retention still holds
Personal cron, if available	A timestamp in the log, and only the marked entry removed afterwards

Write what you did not test: a source that changes during the backup, hostile archives, real scheduling.

Plus: a practice cron job

```
student@server: ~/oslab-work/lab10

$ lab10-cron status
$ lab10-cron install
$ crontab -l
$ lab10-cron remove
$ lab10-cron status
```

The helper adds one marked entry of your own that appends a timestamp every minute. It does not run your backup.

Wait up to two minutes for cron.log, then remove it. Write what you saw, or that cron is not available.

PLUS

Careful

Never edit /etc/crontab, and never remove the whole crontab.

First watch the instructor run the script from / with env -i and with a deliberate mistake: the relative path project. Save your answers in evidence/plus.txt. Plus and Challenge are not needed for full marks.

Challenge: a cron line, as text

student@server: ~/oslab-work/lab10

```
$ # minute hour day-of-month month day-of-week command
$ 30 2 * * * /full/path/to/backup.sh
```

A cron entry has five time fields. Write the line that would run your backup.sh every day at 02:30, with the full path to the script. Do not install it.

CHALLENGE

Answer in evidence/plus.txt

Why does an entry for one calendar date still run again next year?

LIVE CHECKPOINT

Show what you can do alone

Fifteen minutes near the end of the lab. Worth 3 of the 10 points.

Why is there a checkpoint?

It shows what you can do alone

For most of the lab you may use hints and AI tools. The checkpoint is the one moment without them.

The number is new

The rule uses a different number to keep, made for you at that moment. An answer copied from a neighbour does not fit.

The instructor sees who needs help

Your answers arrive at once, so a common mistake can be explained before the next lab.

It is fair

Everyone gets the same kind of task, with their own number.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab10
Read the new rule, answer four questions

→

3. The file task
Copy your script, change keep= only

→

4. oslab check lab10
A fifth milestone must pass

Before it opens

The command says that the checkpoint is not open yet. Wait for the instructor.

While you work

Use your terminal and what you learned in this lab. No AI tools, and no help from a neighbour.

When you finish

oslab check lab10 shows whether the fifth milestone passes. It fails if your retention has the old number written into its logic.

What the checkpoint asks

Part	What you do
Four questions	Read the new rule, with a different number to keep. Answer in the terminal.
The file task	Copy your script, change only the keep= line, run the copy several times, and save the list of archives in checkpoint.txt.
The check	Run oslab check lab10. The fifth milestone, checkpoint, must show PASS.

This is why the retention uses the variable $keep, not a number written into its logic.

DEBRIEF AND SUBMIT

Look back, then hand in

Ten minutes of debrief, ten to submit.

Debrief: look at your prediction again

student@server: ~/oslab-work/lab10

```
$ cat "$OSLAB_WORKSPACE/.records/lab10-predict.json"
$ oslab debrief lab10
```

oslab saved your prediction in a hidden folder, .records. It stays as it was, so the comparison shows which answers you confirmed and which you corrected.

DISCUSS

Class discussion

Your backup worked in your terminal. Which two assumptions would you check before you trust it under cron?

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab10/evidence"
$ cd "$OSLAB_WORKSPACE/lab10"
$ cp -- backup.sh "$SUBMISSION_REPO/lab10/"
$ cp -- evidence/restore.txt evidence/automation.txt "$SUBMISSION_REPO/lab10/evidence/"
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

If you installed the practice cron entry, run lab10-cron remove first, and check with crontab -l. If you did Plus or Challenge, copy evidence/plus.txt the same way.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab10
├── backup.sh
├── evidence
│   ├── automation.txt
│   └── restore.txt
└── README.md

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

Submit 3: commit and push

student@server: ~

```
$ cd ~/os-gic-YOUR_ID
$ git pull
$ git add os-lab-YOUR_ID/lab10
$ git commit -m "Lab 10"
$ git push
```

git pull comes first, in case you pushed from your own computer since.

Replace YOUR_ID with your own ID, as before. The course website checks that the exact file names are there.

How it is marked: 10 points

Evidence for the lab objectives	Points
Script: oslab check lab10 passes the four Core milestones (restorable archive, retention, files that are not yours, restricted environment)	2
Tests: restore.txt and automation.txt show the restore, the retention and the environment test, and the README states what was not tested	2
Prediction saved in time, and corrected with oslab debrief lab10	2
Live checkpoint: the answers and your copy keeping the new number	3
Complete submission: the required files in your repository, on time	1

Every row is marked from your oslab records and the files in your repository, so run oslab check lab10 until it shows what you expect. Your prediction earns its first point for being made, not for being right. Plus and Challenge are not needed for full marks.

Common mistakes

Mistake	What you see	Fix
rm $(ls backups)	Unrelated files removed	Sort the matching names, quote every path
Relative paths in the script	It works in your shell, fails in cron	Find the script's own folder first
Trusting a backup you never restored	A corrupt or incomplete archive	Extract it and compare with cmp
The number written into the logic	The checkpoint copy keeps the old number	Use $keep everywhere, change only keep=
Leaving the practice cron entry	A job that keeps running after the lab	lab10-cron remove, then check crontab -l

BEYOND THE CORE

Schedule it, rotate logs, a health snapshot

Optional extensions. They need your own cron and dates the instructor gives you.

Lab 10

Schedule the repaired backup

student@server: ~/oslab-work/lab10

```
$ crontab -l
$ */5 * * * * /bin/bash /home/student/oslab-work/lab10/backup.sh >> /home/student/oslab-work/lab10/cron.log 2>&1
```

Install it only after backup.sh works under the bare environment. Use absolute paths for the shell, the script and the log.

2>&1 sends error messages to the log too, so you can see why a run failed. Remove the entry when you have seen it run.

Logs, a health snapshot, a guarded date job

LOGS

Rotate your own logs

Move an owned log to a dated name and start a new one, but never while a job is writing it.

HEALTH

A health snapshot

A small script that saves df -h, uptime and the newest backup name to one file, and is safe to run again.

DATES

A guarded date job

Cron has no year. Use dates and the Asia/Phnom_Penh time zone from the instructor, and make the script check the date itself.

Check which time zone the cron daemon on the server uses. Changing TZ in your shell does not change how the schedule is read.

Key takeaways

01

Archive, then compress

tar joins the files. gzip shrinks the bytes. -C keeps the paths inside the archive stable.

02

Restore to prove it

A backup is a backup when you have extracted it and compared the bytes.

03

Keep the scope narrow

Retention touches only the names this script made, in your own folder.

04

cron is a bare shell

Absolute paths, no assumptions about your folder, and test with env -i first.

Next session



COMING UP

Lab 11 (bonus): disk management utilities

Look at disks, partitions and filesystems, with tools that only read.



SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapter 14, File-System Implementation.